Announcements

Topic 6: Searching Linear Structures

By: Professor Lynam

Sequential (Linear) Search

- Scans each element in order of index to find a specific element. (Typically returns false or none/null upon failure)
- Best Case: O(1) – if the target is at list[0]
- Worst Case: O(n) – if the target is at list[len-1] or if the target is not found
- Average Case (assuming only successful searches): when all targets are equally likely. Let's sum up the number of comparisons we have to make at each index.
- If the target is at index 1, just 1 comparison, if at 2, then 2 comparisons… if at n, then n comparisons

Sequential (Linear) Search

- To the find the average of all of these cases, since each is equally likely, and there are n cases to consider, we can just add each case's comparisons all up and divide by the total number of cases, n:
- $\frac{1+2+\cdots+n}{n} = \frac{\sum_1^n i}{n}$
- $\frac{\sum_1^n i}{n} = \frac{\frac{n*(n+1)}{2}}{n} = \frac{n+1}{2}$ which is Big-O of…?
- O(n)
- Which is obvious, but it is good to establish what we mean by the "average" search case

Binary Search

- Binary search needs, ordered data and a direct-access data structure

0	1	2	3	4	5	6	7	8	9	10
2	6	7	14	19	21	36	42	61	77	81

- Target: 14

Probe	Low	High	Mid	Key Comparison
1	0	10	5	21 > 14; eliminate top portion
2	0	4	2	7 < 14; eliminate bottom portion
3	3	4	3	14 = 14; return success

Binary Search

- Binary search needs, ordered data and a direct-access data structure

0	1	2	3	4	5	6	7	8	9	10
2	6	7	14	19	21	36	42	61	77	81

- Target: 63

Probe	Low	High	Mid	Key Comparison
1	0	10	5	21 < 63; eliminate bottom portion
2	6	10	8	61 < 63; eliminate bottom portion
3	9	10	9	77 > 63; eliminate top portion
4	9	8	-	High and low crossed; return failure

Binary Search

- Binary search efficiency:

# keys examined:	1	2	3	…	k	$\log_2 x + 1$
# keys in list:	1	3	7	…	$2^k - 1$	x

- A different perspective… # of probes per index. Let n = 15. Let's see how many probes it takes to find any given value in the list.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
4	3	4	2	4	3	4	1	4	3	4	2	4	3	4

- So, assuming all keys are equally likely to be searched, over 50% of the time we're performing the maximum number of probes!

Binary Search

- Given those perspectives, the analysis is straight-forward:
- Best Case: O(1), when the target is at the midpoint
- Worst Case: O($\log_2 n$), for the maximum number of probes
- Average Case: O($\log_2 n$), since ½ of the possible targets are the worst-case

Binary Search

- Binary search is pretty efficient! Can it be improved though?
- Yes, *if* the data cooperates. We'll need:

1. Binary Search's preconditions (ordered data, direct-access structure)
2. A *lot* of data to be searched
3. Data that is uniformly-distributed (no clusters or gaps)

- Those additional conditions allow us to interpolate a target's likely location within the data, leading to…

Interpolation Search

- Suppose we need to find 780 with the array below:

	0 (low)	1	2		77		99 (high)
List:	10	20	30	…	780	…	1000

- 780 is $\frac{780-10}{1000-10} = \frac{7}{9}$ths into the data's range (10 to 1000).
- Let's guess that 780's location is $\frac{7}{9}$ths into the range of indices, starting from the low index of that range:
- probe = $low + \left\lfloor \frac{target - list[low]}{list[hi] - list[low]} * (high - low) \right\rfloor$
- $0 + \left\lfloor \frac{780-10}{1000-10} * (99 - 0) \right\rfloor = 77$

Interpolation Search

- What about when the data's distribution isn't *perfectly* uniform?
- For example, suppose we have list[0…9999], keys 5…95,000, and target is 85,000. What would our first probe be?
- First probe: $0 + \left\lfloor \frac{85000-5}{95000-5} * (9999 - 0) \right\rfloor = 8946$
- Suppose we find 86,500 at list[8946]. 85,000 is smaller, so we just slide the 'high' down to index 8945.
- Assuming list[8945]=86487, what would our second probe be?
- $0 + \left\lfloor \frac{85000-5}{86487-5} * (8945 - 0) \right\rfloor = 8791$
- Just continue the search like this until success or failure!

Interpolation Search

- Interpolation search is just binary search with a different formula for selecting probe indices… but the efficiency analysis is not the same
- Best Case: O(1), when the target is at list[probe]
- Worst Case: O(n), when the data is badly distributed!
- Average Case: O($\log_2 (\log_2 n)$); it's a very complex analysis
- Is interpolation search even worth it, given the worst case is the same as sequential search?

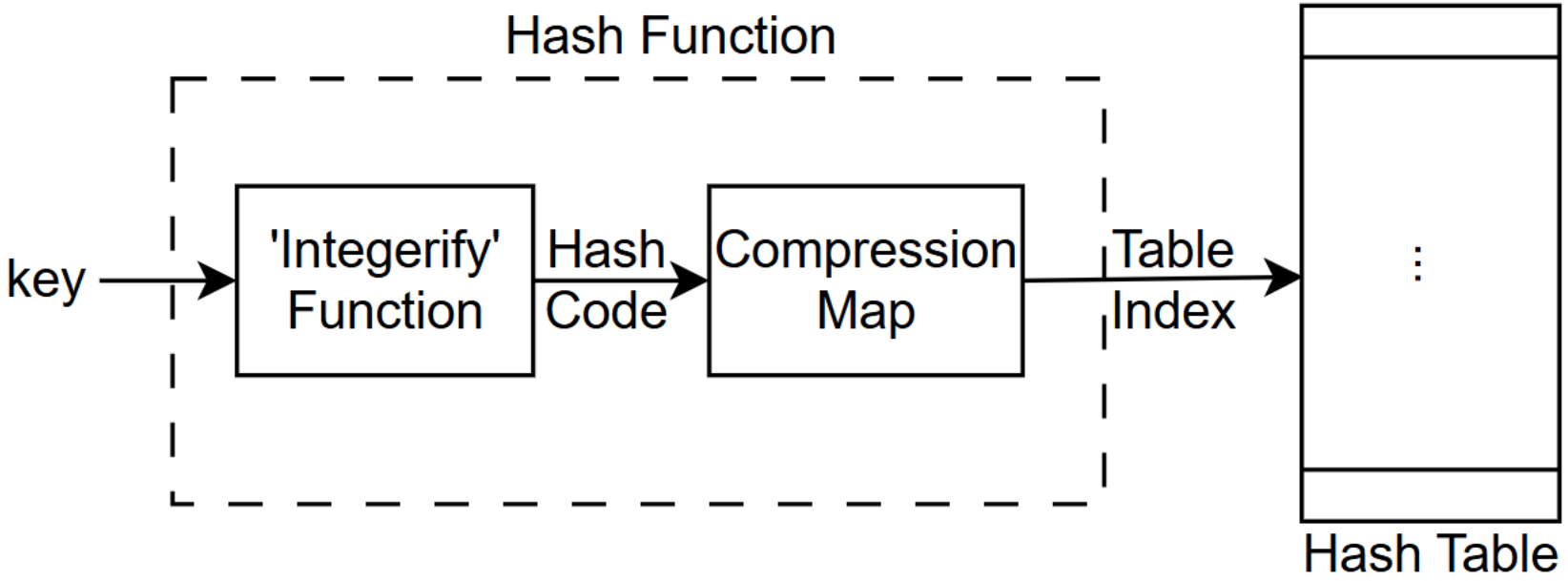
N=	2^5	2^{10}	2^{20}	2^{30}	2^{40}
$\log_2 n$	5	10	20	30	40
$\log_2 (\log_2 n)$	2.3	3.3	4.3	4.9	5.3

Introduction to Hashing

- Ideally, what would we want our search big-O to always be?
- O(1)
- …which we cannot achieve in all cases, but that doesn't mean that we shouldn't try! A way that we've tried to achieve this is via *hashing*

Introduction to Hashing

- A general view of hashing:
- ‘Integerify’ Function: needed when key isn’t already an integer
- Compression Map: brings hash code into the table’s index range
- The hashcode() method in Java’s Object class is an ‘Integerify’ function



Introduction to Hashing

- The goal of a hash function is to produce well-distributed table indices (like a random number generator, but reproducible)
- There are three classic hash function techniques we'll be covering:

1. Division Method
2. Multiplication Method
3. The 'MAD' Method (Multiplication, Addition, Division)

Introduction to Hashing

- The Division Method:
- Assuming that the hash table has *m* slots:
- $h(key) = key \,\%\, m$
- Considerations when selecting the table size (*m*):
- Available main memory
- Load factor, aka, $\frac{\#\ of\ keys}{table\ size}$ a ratio that, as it approaches 1, causes "collisions"
- Data patterns, such as choosing *m* to be relatively prime to data (consider a bunch of even-numbered data with an even-numbered *m*)

Introduction to Hashing

- The Multiplication Method:
- Why not just stick with division? Divisions (%) are relatively slow to compute. So, why not use a similar operation (multiplication) that is more efficient?
- $h(key) = \lfloor m * (key * A - \lfloor key * A \rfloor) \rfloor$, where m is the hash table size, and A is a constant such that 0 < A < 1.
- This hash function works like this: $key * A - \lfloor key * A \rfloor$ will create a value [0, 1). Multiplying that by m expands that value to the range of the hash table's indices.

Introduction to Hashing

- The ‘MAD’ Method:
- Addresses another issue with the Division Method: we don’t know that m will be a good choice to randomize the indices for the Division Method. So, why not mix up the keys a bit first?
- $h(key) = |a * key + b| \ \% \ m$, where a, b ∈ Z^+ and m and a are relatively prime
- This has the same form as a basic linear congruential function used to produce pseudorandom numbers: $X_{n+1} = (a * X_n + c) \ \% \ m$

Collision Resolution Strategies

- A collision occurs when two key values hash to the same table index. There are two primary ways of dealing with this:

1. Open Addressing
2. Chaining

Open Addressing

- Open Addressing: Place the key in the ‘next’ available location within the hash table
- The meaning of ‘next’ depends on the flavor of open addressing used; we’ll discuss a few (linear/quadratic probing, and double hashing)
- The table is treated as circular for the purposes of open addressing; if ‘next’ is off the bottom of the table, just wrap back around to the top

Linear Probing

- The ‘next’ location is just the next larger index (wrapping around if necessary)
- All we need to do to implement this is to ‘wrap’ the hash function with another function:
- $lp(key, i) = (h(key) + i)\ \%\ m$
- We parameterize *i* to make walking down the table by any number of indices easy to express

Linear Probing

- Linear probing general function: $lp(key, i) = (h(key) + i)\ \%\ m$
- Let's do a specific example. Let $h(key) = key\ \%\ 7$. What would *lp()* look like?
- $lp(key, i) = (key\ \%\ 7 + i)\ \%\ 7$
- Insert 13, 25, 34, 5, and 19 into the hash table.

0	1	2	3	4	5	6
34	19			25	5	13

lp(13, 0) = 6
lp(25, 0) = 4
lp(34, 0) = 6 (collision!)
lp(34, 1) = 0
lp(5, 0) = 5

lp(19, 0) = 5 (collision!)
lp(19, 1) = 6 (collision!)
lp(19, 2) = 0 (collision!)
lp(19, 3) = 1 (finally!)
Looks awfully like linear search…

Linear Probing

- Linear probing general function: $lp(key, i) = (h(key) + i)\ \%\ m$
- Let's do a specific example. Let $h(key) = key\ \%\ 7$
- $lp(key, i) = (key\ \%\ 7 + i)\ \%\ 7$
- Searching: Find 34. Then Find 42.

0	1	2	3	4	5	6
34	19			25	5	13

lp(34, 0) = 6, but array[6] = 13, not 34. Have to move on!
lp(34, 1) = 0 Success!
lp(42, 0) = 0, but array[0] = 34, not 42. Move on!
lp(42, 1) = 1, but array[1]=19, not 42. Move on!
lp(42, 2) = 2, which is unused, which means 42 is not in the table